

Module 6

IEEE 1800-2017 (Current SystemVerilog)

Learning objectives

- Master the current SystemVerilog standard (IEEE 1800-2017): unified LRM, merged Verilog and SystemV...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["IEEE 1800-2017 Context"]

T2["Verilog as a Subset of 1800-"]

T1 --> T2

T3["Design Constructs in 1800-20"]

T2 --> T3

Master the current SystemVerilog standard (IEEE 1800-2017): unified LRM, merged Verilog and SystemVerilog, and design/verification overview.

Overview

- This module covers IEEE Std 1800-2017, the current SystemVerilog standard. In 1800-2017, the Verilo...

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics
Covered set the IEEE scope for Module 6.
- Study design architecture — See how DUTs, examples, and testbenches fit together under ``module6/``.
- Work through labs in order — Open ``module6/EXAMPLES.md`` and run each ``make clean && make run`` from...
- Run the full module script — ``./scripts/module6.sh`` from the repo root to simulate all examples and...
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at...

Suggested learning path (2/2)

- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use ``module6/CHECKLIST.md`` before starting the next module.

Design architecture

1. Repository layout (module6/)

- IEEE 1800-2017 — current unified SystemVerilog LRM
- Summarizes design subset plus 2017 language updates
- module6/examples/ — version summary, migration, assertions recap
- scripts/module6.sh — current-standard lab sweep

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
subgraph DUT["module6/dut — 1800-2017"]
  ALU["alu_2017 logic+always_comb"]
  BLK["design_block_2017"]
  FSM["fsm_2017 unique case"]
end
```

2. 2017 standard in practice

- Verilog is a subset of SystemVerilog in the 2017 LRM
- logic, always_comb/ff, packages, interfaces — production RTL baseline
- unique case, priority case — prefer over vendor attributes
- Course maps 2017 features back to Modules 4–5 introductions

```
Verification Architecture
Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
  TB["test_alu_2017.sv"] --> ALU
  TB2["future block/fsm tests"] --> BLK
  LINT["synth/lint flags 2017"] --> DUT["module6/dut"]
```

3. Single-driver and type discipline

- logic enforces one driver per net in synthesizable RTL
- Package import :: scope — shared enums and parameters
- See rtl_architecture.png — same DUT/TB split as earlier modules

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M6["module6.sh"] --> ALU["test_alu_2017 directed ops"]

M6 --> MAKE["module6/tests make"]

ALU --> DOC["document project standard 2017"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph DUT["module6/dut — 1800-2017"]

ALU["alu_2017 logic+always_comb"]

BLK["design_block_2017"]

FSM["fsm_2017 unique case"]

end

Module 6: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

TB["test_alu_2017.sv"] --> ALU

TB2["future block/fsm tests"] --> BLK

LINT["synth/lint flags 2017"] --> DUT["module6/dut"]

Module 6: stimulus, observation, and checking.

logic single-driver pattern

```
/*
 * IEEE 1800-2017: logic and single driver (2017 clarified)
 * logic must have exactly one driver; comb and seq in separate blocks.
 */

module comb_block (
    input  logic [7:0] a,
    input  logic [7:0] b,
    output logic [7:0] sum,
    output logic      eq
);
    assign sum = a + b;
    always_comb eq = (a == b);
endmodule

module seq_block (
# ...
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module6/examples/<example>/ — run make run from that directory

Key files to study

Open these in the repo

- `module6/examples/subsets/`
- `module6/examples/design_recap/`
- `module6/examples/version_summary/`
- `scripts/module6.sh`

Verification & testing methods

1. Execution flow

- make run per lab;
./scripts/module6.sh for full module
- version_summary example prints construct-by-standard table

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M6["module6.sh"] --> ALU["test_alu_2017 directed ops"]

M6 --> MAKE["module6/tests make"]

ALU --> DOC["document project standard 2017"]

2. Regression before Module 7

- Confirm all 2017-subset examples compile with your simulator
- Review CHECKLIST.md — ready to compare versions in Module 7

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M6["module6.sh"] --> ALU["test_alu_2017 directed ops"]

M6 --> MAKE["module6/tests make"]

ALU --> DOC["document project standard 2017"]

Version summary runner

```
/*  
 * IEEE 1800-2017: Version timeline summary (1995 -> 2017)  
 * Small design using 2017 subset; display reminds which standard added  
what.  
 */  
  
module mux_2017 (  
    input  logic [1:0] sel,  
    input  logic [7:0] a,  
    input  logic [7:0] b,  
    input  logic [7:0] c,  
    input  logic [7:0] d,  
    output logic [7:0] y  
);  
    always_comb begin  
        unique case (sel)  
            2'b00: y = a;
```

Syllabus topics

Protocol & design details

Detail: Unified LRM (2017)

- Before 2017: IEEE 1364 defined Verilog; IEEE 1800 defined SystemVerilog (extending 1364). Two documents.
- 1800-2017: One LRM (IEEE 1800-2017) defines both the Verilog subset and SystemVerilog. IEEE 1364 is obsolete.
- Implication: When you say "IEEE 1800-2017," you are referring to the full language (Verilog + SystemVerilog).

Detail: What 1800-2017 Adds or Clarifies Over 2012 (1)

- Structure: Single LRM; consistent numbering and cross-references; Verilog and SystemVerilog in one...
- Clarifications: Scheduling, generate, interfaces, assertions, coverage, DPI, and many other areas r...
- Corrections: Errata and ambiguous cases addressed.
- Additions: Any new or clarified features in the 2017 edition (check LRM for specifics; often increm...

Detail: What 1800-2017 Adds or Clarifies Over 2012 (2)

- No separate 1364: For new work, use 1800-2017;
 legacy "1364-only" is the subset of 1800 that omits...

Detail: Backward Compatibility

- RTL and testbenches written for 1364-1995 through 1800-2012 remain valid in 1800-2017 when they use...
- Adopt 2017 as the reference standard for new projects unless a project or tool explicitly targets a...

Detail: Verilog Subset (Design)

- Modules, ports: ANSI or non-ANSI (1364-1995 style);
wire, reg.
- Procedural blocks: always, initial; always @*
(2001); explicit sensitivity (1995).
- No SystemVerilog-only: No logic,
always_comb/always_ff, interfaces, packages,
unique/priority case...
- Use: When the flow or legacy code requires "Verilog
only" (e.g. synthesis or IP that does not accep...

Detail: SystemVerilog Design Subset (1800) (1)

- logic, always_comb, always_ff, always_latch.
- Interfaces and modports.
- Packages and import.
- unique case, priority case.

Detail: SystemVerilog Design Subset (1800) (2)

- 2-state types (bit, int, etc.) where useful.
- Use: When the flow supports SystemVerilog; preferred for new RTL for clarity and tool checking.

Detail: Choosing a Subset

- Verilog-only (1364 subset): Maximum portability; use when tools or IP are Verilog-only.
- SystemVerilog design (1800 subset): Better expressiveness and checking; use when the flow supports...

Detail: logic and Single Driver

- Rule: logic must have exactly one driver (one assign, one always block, or one port connection).
- 2017: Semantics of multiple drivers, resolution, and connection rules are clarified in the LRM.

Detail: `always_comb`, `always_ff`, `always_latch`

- Intent: Combinational, sequential (flip-flop), or latch; tools can check and warn.
- 2017: Execution order, sensitivity, and interaction with other processes are clarified.

Detail: Interfaces and Modports

- Use: Bundle signals; modports define direction per role.
- 2017: Interface connection, parameterization, and modport rules are refined.

Detail: Packages and import

- Use: Shared types, constants, functions; explicit namespace.
- 2017: Import scope, search order, and compilation-unit interaction are clarified.

Detail: unique case and priority case

- Semantics: At most one match (unique) or first match wins (priority); runtime checks possible.
- 2017: Case statement semantics and synthesis mapping are clarified.

Detail: Immediate and Concurrent Assertions

- Immediate: ``assert (condition);`` — executed when reached; useful for invariants in RTL.
- Concurrent: ``assert property (...);`` — temporal; used in verification and bind files.
- 2017: Property syntax, scheduling, and binding are clarified.

Detail: Checkers

- Checker: Reusable block of assertions/assumptions/coverage (from 2012); instantiated or bound.
- 2017: Checker semantics and instantiation are clarified.

Detail: Coverage

- Covergroups, coverpoints: Defined in 1800; used in verification.
- 2017: Coverage options and semantics are refined.

Detail: Synthesizable Subset

- Generally supported: logic, always_comb, always_ff, interfaces, packages, generate, parameter, loca...
- Often not supported or tool-specific: Classes, full SVA, dynamic arrays/queues in RTL, virtual inte...
- Check tool docs: Each vendor defines the exact synthesizable subset for 1800-2017.

Detail: Simulation Support

- 1800-2017: Most commercial and several open-source simulators support a large subset of 1800-2017.
- Revisions: Tools may support 2005, 2009, 2012, or 2017; choose project standard to match tool and I...

Detail: Linting and Formal

- Lint: Many tools check 1800-2017 syntax and common RTL rules (e.g. single driver, blocking/nonblock...
- Formal: Assertions and checkers are used with formal tools; 2017 clarifies semantics for consistenc...

Detail: Choosing a Standard

- Legacy or Verilog-only flow: Use 1364-2005 subset (no SystemVerilog) or document "Verilog as per 18...
- New RTL with SystemVerilog: Use 1800-2017 (or 2012/2009 if tool lags); adopt logic, always_comb/alw...
- Verification: Use 1800-2017 (or supported revision) for SVA, checkers, coverage, classes.
- Single revision: Prefer one stated revision (e.g. "1800-2017") for the project to avoid mixed seman...

Detail: Migration (Brief)

- 1364 → 1800 design: Replace wire/reg with logic (single driver); always @* → always_comb; always @(...
- Older 1800 → 2017: Most code remains valid; adopt 2017 clarifications and any new features the proj...
- Tool support: Confirm simulator and synthesis support for the chosen revision before locking the st...

Verification flow (reference)

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M6["module6.sh"] --> ALU["test_alu_2017 directed ops"]

M6 --> MAKE["module6/tests make"]

ALU --> DOC["document project standard 2017"]

Stimulus, DUT response, and check points for this module.

1. IEEE 1800-2017 Context (1/2)

- Before 2017: IEEE 1364 defined Verilog; IEEE 1800 defined SystemVerilog (extending 1364). Two documents.
- 1800-2017: One LRM (IEEE 1800-2017) defines both the Verilog subset and SystemVerilog. IEEE 1364 is obsolete.
- Implication: When you say "IEEE 1800-2017," you are referring to the full language (Verilog + SystemVerilog).
- Structure: Single LRM; consistent numbering and cross-references; Verilog and SystemVerilog in one document.
- Clarifications: Scheduling, generate, interfaces, assertions, coverage, DPI, and many other areas redefined.
- Corrections: Errata and ambiguous cases addressed.

1. IEEE 1800-2017 Context (2/2)

- Additions: Any new or clarified features in the 2017 edition (check LRM for specifics; often increm...
- No separate 1364: For new work, use 1800-2017; legacy "1364-only" is the subset of 1800 that omits...
- RTL and testbenches written for 1364-1995 through 1800-2012 remain valid in 1800-2017 when they use...
- Adopt 2017 as the reference standard for new projects unless a project or tool explicitly targets a...

2. Verilog as a Subset of 1800-2017 (1/2)

- Modules, ports: ANSI or non-ANSI (1364-1995 style);
wire, reg.
- Procedural blocks: always, initial; always @*
(2001); explicit sensitivity (1995).
- No SystemVerilog-only: No logic,
always_comb/always_ff, interfaces, packages,
unique/priority case...
- Use: When the flow or legacy code requires "Verilog
only" (e.g. synthesis or IP that does not accep...
- logic, always_comb, always_ff, always_latch.
- Interfaces and modports.

2. Verilog as a Subset of 1800-2017 (2/2)

- Packages and import.
- unique case, priority case.
- 2-state types (bit, int, etc.) where useful.
- Use: When the flow supports SystemVerilog; preferred for new RTL for clarity and tool checking.
- Verilog-only (1364 subset): Maximum portability; use when tools or IP are Verilog-only.
- SystemVerilog design (1800 subset): Better expressiveness and checking; use when the flow supports...

3. Design Constructs in 1800-2017 (Recap and Clarifications) (1/2)

- Rule: logic must have exactly one driver (one assign, one always block, or one port connection).
- 2017: Semantics of multiple drivers, resolution, and connection rules are clarified in the LRM.
- Intent: Combinational, sequential (flip-flop), or latch; tools can check and warn.
- 2017: Execution order, sensitivity, and interaction with other processes are clarified.
- Use: Bundle signals; modports define direction per role.
- 2017: Interface connection, parameterization, and modport rules are refined.

3. Design Constructs in 1800-2017 (Recap and Clarifications) (2/2)

- Use: Shared types, constants, functions; explicit namespace.
- 2017: Import scope, search order, and compilation-unit interaction are clarified.
- Semantics: At most one match (unique) or first match wins (priority); runtime checks possible.
- 2017: Case statement semantics and synthesis mapping are clarified.

4. Assertions and Verification in 1800-2017 (Context) (1/2)

- Immediate: ``assert (condition);`` — executed when reached; useful for invariants in RTL.
- Concurrent: ``assert property (...);`` — temporal; used in verification and bind files.
- 2017: Property syntax, scheduling, and binding are clarified.
- Checker: Reusable block of assertions/assumptions/coverage (from 2012); instantiated or bound.
- 2017: Checker semantics and instantiation are clarified.
- Covergroups, coverpoints: Defined in 1800; used in verification.

4. Assertions and Verification in 1800-2017 (Context) (2/2)

- 2017: Coverage options and semantics are refined.

5. Synthesis and Tool Support (1800-2017) (1/2)

- Generally supported: logic, always_comb, always_ff, interfaces, packages, generate, parameter, loca...
- Often not supported or tool-specific: Classes, full SVA, dynamic arrays/queues in RTL, virtual inte...
- Check tool docs: Each vendor defines the exact synthesizable subset for 1800-2017.
- 1800-2017: Most commercial and several open-source simulators support a large subset of 1800-2017.
- Revisions: Tools may support 2005, 2009, 2012, or 2017; choose project standard to match tool and I...
- Lint: Many tools check 1800-2017 syntax and common RTL rules (e.g. single driver, blocking/nonblock...

5. Synthesis and Tool Support (1800-2017) (2/2)

- Formal: Assertions and checkers are used with formal tools; 2017 clarifies semantics for consistenc...

6. Version Timeline Summary (1995 → 2017)

- Design migration: 1364-1995 → 2001 → 2005 (Verilog);
then 1800-2005 → 2009 → 2012 → 2017 (SystemVer...)

7. Choosing a Standard and Migration (1/2)

- Legacy or Verilog-only flow: Use 1364-2005 subset (no SystemVerilog) or document "Verilog as per 18...
- New RTL with SystemVerilog: Use 1800-2017 (or 2012/2009 if tool lags); adopt logic, always_comb/alw...
- Verification: Use 1800-2017 (or supported revision) for SVA, checkers, coverage, classes.
- Single revision: Prefer one stated revision (e.g. "1800-2017") for the project to avoid mixed seman...
- 1364 → 1800 design: Replace wire/reg with logic (single driver); always @* → always_comb; always @(...
- Older 1800 → 2017: Most code remains valid; adopt 2017 clarifications and any new features the proj...

7. Choosing a Standard and Migration (2/2)

- Tool support: Confirm simulator and synthesis support for the chosen revision before locking the st...

8. 1800-2017 as the Single Reference (1/5)

- One LRM: For any question on Verilog or SystemVerilog syntax/semantics, 1800-2017 is the single ref...
- Subsets: Define project subsets explicitly (e.g. "1800-2017 design subset, no SVA in RTL") so that...
- Future revisions: New IEEE 1800 revisions (e.g. 2024+) will extend 2017; the version-centric approach...
- Same small design: one file using only 1364-style (wire/reg, always @*), one using 1800 design (log...
- Key Concepts: 1364 subset vs 1800 design subset within 1800-2017
- One module using logic, always_comb, always_ff, package, unique case (all 1800-2017 compliant)

8. 1800-2017 as the Single Reference (2/5)

- Key Concepts: Current standard design subset in one place
- Simple invariant (e.g. one-hot state) with assert in design
- Key Concepts: Synthesizable RTL + assertion; 2017 semantics
- Table or script that lists which construct belongs to which standard (1995-2017)
- Key Concepts: Full version timeline; choosing a standard
- Same 4:1 mux in 1364-2005 style vs 1800-2017 style (wire/reg→logic, always @*→always_comb, unique c...

8. 1800-2017 as the Single Reference (3/5)

- Key Concepts: Step-by-step migration from 1364 to 1800 design subset
- Comb and seq blocks; each logic has exactly one driver (2017 clarified)
- Key Concepts: Single driver for logic; assign, always_comb, always_ff
- Priority encoder using priority case (first match wins)
- Key Concepts: priority case; 2017 semantics for encoders/arbiters
- Shared types and parameters in package; import in module and top

8. 1800-2017 as the Single Reference (4/5)

- Key Concepts: Package and import; 2017 scope and search order clarifications
- Decoder with unique case and default (no latch)
- Key Concepts: unique case; at most one match; 2017 clarified
- alu_2017.sv: Small ALU with logic, always_comb, package type, optional immediate assertion
- Example: Current standard design subset; no legacy 1364-only constructs
- design_block_2017.sv: Block using 1800-2017 design constructs (logic, always_comb/always_ff, packag...

8. 1800-2017 as the Single Reference (5/5)

- Example: Interface-style or block-level connectivity; 2017 design subset
- fsm_2017.sv: FSM with unique case and one immediate assertion (state encoding)
- Example: 1800-2017 RTL; assertion for invariant
- test_alu_2017.sv: ALU testbench (1800-2017 compatible)
- Key Features: Exercises `dut/alu\_2017.sv`; no use of deprecated or non-2017 constructs

Hands-on examples

Module 6 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module6_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Example 1: Same small design: one file using only 1364-style (wire/re...

- 1364 subset vs 1800 design subset within 1800-2017

Demo: Same small design: one file using only 1364-style (wire/reg, al...

```
$ cd module6/examples/subsets && make clean && make run
```

Terminal: ex01_subsets

```
rm -f top
iverilog -g2012 -o top verilog_vs_sv.sv
vvp top
1800-2017: Verilog subset vs SystemVerilog subset
  sel=0 y1364=0 y1800=0
  sel=1 y1364=1 y1800=1
  Same behavior; 1800-2017 LRM defines both.
verilog_vs_sv.sv:43: $finish called at 20 (1s)
```

Example 2: One module using logic, always_comb, always_ff, package, u...

- Current standard design subset in one place

Demo: One module using logic, always_comb, always_ff, package, unique...

```
$ cd module6/examples/design_recap && make clean && make run
```

Terminal: ex02_design_recap

```
rm -f top
iverilog -g2012 -o top design_recap.sv
vvp top
1800-2017 design_recap: logic, always_comb, always_ff, package, unique case
  count= 4 mux_sel=10 y= 4
design_recap.sv:68: $finish called at 60 (1s)
design_recap.sv:36: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 3: Simple invariant (e.g. one-hot state) with assert in design

- Synthesizable RTL + assertion; 2017 semantics

Demo: Simple invariant (e.g. one-hot state) with assert in design

```
$ cd module6/examples/assertions && make clean && make run
```

Terminal: ex03_assertions

```
rm -f top
iverilog -g2012 -o top immediate_in_rtl.sv
vvp top
1800-2017: Immediate assertion in RTL (one-hot invariant)
  req=001 grant=001
  req=110 grant=010 (priority)
  req=000 grant=000
Assertion passes when grant is one-hot or zero.
immediate_in_rtl.sv:36: $finish called at 30 (1s)
immediate_in_rtl.sv:11: sorry: constant selects in always_* processes are not currently supported
immediate_in_rtl.sv:11: sorry: constant selects in always_* processes are not currently supported
immediate_in_rtl.sv:11: sorry: constant selects in always_* processes are not currently supported
immediate_in_rtl.sv:21: warning: System task ($error) cannot be synthesized in an always_comb process
```

Example 4: Table or script that lists which construct belongs to whic...

- Full version timeline; choosing a standard

Demo: Table or script that lists which construct belongs to which sta...

```
$ cd module6/examples/version_summary && make clean && make run
```

Terminal: ex04_version_summary

```
rm -f top
iverilog -g2012 -o top version_summary.sv
vvp top
1800-2017: Version timeline (1364-1995 -> 1800-2017)
  1364-1995: wire/reg, always, initial
  1364-2001: ANSI ports, always @*, generate
  1364-2005: last Verilog-only
  1800-2005: logic, always_comb/always_ff, interfaces, packages
  1800-2009/2012: inside/==?, checkers, array methods
  1800-2017: unified LRM (Verilog + SystemVerilog in one standard)
sel=11 y=08
version_summary.sv:41: $finish called at 10 (1s)
version_summary.sv:15: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 5: Same 4:1 mux in 1364-2005 style vs 1800-2017 style (wire/r...

- Step-by-step migration from 1364 to 1800 design subset

Demo: Same 4:1 mux in 1364-2005 style vs 1800-2017 style (wire/reg→lo...

```
$ cd module6/examples/migration && make clean && make run
```

Terminal: ex05_migration

```
rm -f top
iverilog -g2012 -o top migration.sv
vvp top
1800-2017: Migration 1364 -> 1800 design subset
  sel=10 y1364=04 y1800=04 (same)
  Steps: wire/reg->logic, always @*->always_comb, add unique case
migration.sv:52: $finish called at 10 (1s)
migration.sv:30: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 6: Comb and seq blocks; each logic has exactly one driver (20...

- Single driver for logic; assign, always_comb, always_ff

Demo: Comb and seq blocks; each logic has exactly one driver (2017 cl...

```
$ cd module6/examples/logic_single_driver && make clean && make run
```

Terminal: ex06_logic_single_driver

```
rm -f top
iverilog -g2012 -o top logic_single_driver.v
vvp top
1800-2017: logic single driver (2017 clarified)
  comb: a= 10 b= 20 sum= 30 eq=0
  seq:  d= 55 q= 55
  Each logic has one driver: assign, always_comb, or always_ff.
logic_single_driver.v:50: $finish called at 50 (1s)
```

Example 7: Priority encoder using priority case (first match wins)

- priority case; 2017 semantics for encoders/arbiters

Demo: Priority encoder using priority case (first match wins)

```
$ cd module6/examples/priority_case && make clean && make run
```

Terminal: ex07_priority_case

```
rm -f top
iverilog -g2012 -o top priority_case.sv
vvp top
1800-2017: priority case (first match wins)
  in=0100 enc=10 valid=1 (bit 2)
  in=0011 enc=00 valid=1 (bit 0 wins)
  in=0000 enc=00 valid=0 (none)
priority_case.sv:36: $finish called at 30 (1s)
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
priority_case.sv:11: sorry: constant selects in always_* processes are not currently supported (
```

Example 8: Shared types and parameters in package; import in module a...

- Package and import; 2017 scope and search order clarifications

Demo: Shared types and parameters in package; import in module and top

```
$ cd module6/examples/package_import && make clean && make run
```

Terminal: ex08_package_import

```
rm -f top
iverilog -g2012 -o top package_import.sv
vvp top
1800-2017: package and import (2017 clarified)
  a= 40 b= 60 sum=100
  Shared types/params in package; import where needed.
package_import.sv:33: $finish called at 10 (1s)
```

Example 9: Decoder with unique case and default (no latch)

- unique case; at most one match; 2017 clarified

Demo: Decoder with unique case and default (no latch)

```
$ cd module6/examples/unique_case && make clean && make run
```

Terminal: ex09_unique_case

```
rm -f top
iverilog -g2012 -o top unique_case.sv
vvp top
1800-2017: unique case (at most one match)
  sel=00 y=0001
  sel=10 y=0100
  sel=11 y=1000
  unique case: no overlap; default avoids latch.
unique_case.sv:34: $finish called at 30 (1s)
unique_case.sv:12: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Common pitfalls

Assuming 1364 Is Still a Separate Standard

- Mistake: Citing "IEEE 1364-2005" as the current Verilog standard for new designs.
- Reality: 1364 is merged into 1800-2017; new work should reference 1800-2017 (and use a subset if Ve...
- Correct: Use "1800-2017" as the reference; say "Verilog subset of 1800-2017" when restricting to no...
- Why: Avoids confusion and ensures one LRM for all language questions.

Mixing Revisions in One Project

- Mistake: Using 1800-2017 in one file and 1800-2005-only in another without a clear subset rule.
- Reality: Can lead to tool-dependent behavior or subtle bugs.
- Correct: Define one project standard (e.g. 1800-2017) and a subset (e.g. design only, no SVA in RTL...
- Why: Predictable tool behavior and easier maintenance.

Ignoring Tool Support

- Mistake: Writing 1800-2017 syntax that the synthesis or simulation tool does not support.
- Reality: Compile or elaboration errors; or tool-specific interpretation.
- Correct: Check tool documentation for supported LRM revision and synthesizable subset; restrict cod...
- Why: Ensures portability and correct results.

Assertions in Synthesis

- Mistake: Expecting concurrent assertions or complex checkers to be synthesized.
- Reality: Most assertion/checker logic is for simulation and formal; not for netlist.
- Correct: Use immediate assertions for simple invariants if the tool supports them in RTL; keep heav...
- Why: Synthesis tools typically ignore or strip assertions; RTL should remain synthesizable.

Over-Migrating Legacy Code

- Mistake: Converting all legacy 1364 RTL to 1800 (logic, interfaces, etc.) without need.
- Reality: Risk of introducing bugs; effort may not pay off if the block is stable and Verilog-only i...
- Correct: Migrate when adding features, fixing bugs, or when the project standard moves to 1800; oth...
- Why: Balance benefit (readability, tool checking) with risk and effort.

Practice & assessment

What you should know

- ✓ Describe IEEE 1800-2017 as the unified LRM (Verilog + SystemVerilog in one standard)
- ✓ Explain "Verilog" as a subset of 1800-2017 and when to use Verilog-only vs SystemVerilog design
- ✓ Recap design constructs (logic, always_comb/always_ff, interfaces, packages, unique/priority case...
- ✓ Use immediate assertions in RTL appropriately; understand SVA/checker context
- ✓ Summarize the version timeline (1364-1995 through 1800-2017) and choose a standard for a project
- ✓ Plan simple migration from older Verilog or SystemVerilog to 1800-2017

Exercises

- Subset Comparison
- 1800-2017 Design Checklist
- Immediate Assertion
- Version Table
- Migration Plan

Exercises

- Assuming 1364 Is Still a Separate Standard
- Mixing Revisions in One Project
- Ignoring Tool Support
- Assertions in Synthesis
- Over-Migrating Legacy Code

Summary & next steps

- Master the current SystemVerilog standard (IEEE 1800-2017): unified LRM, merged Verilog and SystemV...
- Complete module6/CHECKLIST.md
- Review module6/EXAMPLES.md and run each lab
- Next: Next module in course